Programming
Human-Centered Artificial Intelligence
Dr. Ramon Winterhalder
A.Y. 2026/27

Exercise sheet 4

Interations – while, for, and the patterns inside every loop

Not graded, not handed in. Do them anyway. This is where the learning happens! Solutions are published after we discuss them in class.

Trace before you run. For every loop on this sheet, draw the table first: one column per variable, one row per pass. It is faster than staring, and it is what the exam asks for.

Part A — while

A1 *Trace four loops*

For each one, draw the trace table and give the **final value of every variable**.

```
# 1                          # 3
i = 0                        x = 5
total = 0                    result = 1
while i < 4:                 while x > 0:
    total = total + i            result = result * x
    i = i + 1                    x = x - 1

# 2                          # 4
n = 20                       i = 0
count = 0                    while True:
while n > 1:                     i = i + 3
    n = n // 2                   if i > 10:
    count = count + 1                break
```

Number 3 computes something with a name. What?

A2 *Never starts, never stops, or runs*

For each loop say which of them *never starts, never stops, or runs* — and if it runs, say **how many times**.

```
# 1                  # 3                  # 5
i = 10               i = 1                i = 5
while i < 5:         while i < 5:         while i > 0:
    print(i)             print(i)             print(i)
    i = i + 1            i = i + 1            i = i - 2

# 2                  # 4                  # 6
i = 1                i = 0                n = 3
while i <= 3:        while i != 10:       while n >= 0:
    print(i)             i = i + 3            print(n)
                                              n = n - 1
```

A3 *The accumulator*

Write a program that asks for a whole number `n` and prints the sum $1 + 2 + \cdots + n$.

a) Write it with a `while` loop.

b) Check it on $n = 4$ (should be 10) and $n = 1$ (should be 1).

c) What does your program print for $n = 0$? Is that what it should print?

Write the comment plan first. Bonus: Do you know how Gauss has solved this problem?

A4 *Keep asking until it is valid*

Write a program that asks for a grade out of 30, and keeps asking until the number typed is between 0 and 30. Then it prints the grade.

Write it twice:

a) with the question written out twice — once before the loop, once inside it;

b) with `while True:` and a `break`.

Which do you find easier to read? There is no official right answer here, but have a reason.

A5 *It runs, and the answer is wrong*

This is supposed to add up the numbers from 1 to 5.

```
i = 1
while i <= 5:
    total = 0
    total = total + i
    i = i + 1

print(total)
```

a) What does it print?

b) What should it print?

c) Which line is wrong, and why does the mistake produce *that* particular answer?

d) Fix it by moving exactly one line.

Part B — for and range

B1 *What does range give you?*

Write out every number each one produces. Two of them produce nothing at all.

```
1.  range(5)
2.  range(2, 7)
3.  range(0, 10, 3)
4.  range(5, 0, -1)
5.  range(3, 3)
6.  range(10, 0)
```

Then: how would you write “the numbers 1 to n, including n”?

B2 *Nested loops*

For each one, say how many lines it prints, and write out the first three.

```
# 1
for i in range(3):
    for j in range(2):
        print(i, j)

# 2
for i in range(3):
    print("row", i)
    for j in range(2):
        print("  col", j)
```

```
# 3
for i in range(1, 4):
    print("*" * i)


# 4
for i in range(2):
    for j in range(i + 1):
        print(j, end=" ")
    print()
```

In number 4, what would change if the bare `print()` were indented one level further?

B3 *Off by one*

Each of these is meant to do what its comment says, and each is wrong by exactly one. Say what it actually does, and fix it.

```
# 1 - the sum of 1 to n
total = 0
for i in range(1, n):
    total = total + i

# 2 - count down from 10 to 1
for i in range(10, 1, -1):
    print(i)

# 3 - print five stars, one per line
for i in range(1, 5):
    print("*")
```

B4 *Two short ones*

a) Ask for a number `n` and count down from `n` to 1, then print `Liftoff`.

b) Ask for a number `n` and print its times table from $n \times 1$ to $n \times 10$, one line each, in the form `7 x 3 = 21`.

B5 *Factorial*

$n! = 1 \times 2 \times 3 \times \cdots \times n$, and $0! = 1$.

Write a program that asks for `n` and prints $n!$.

a) What should the accumulator start at, and why is it not 0?

b) Check it on $n = 5$ (should be 120) and $n = 0$ (should be 1).

B6 *Shapes*

a) Print a left-aligned triangle of stars of height `n`:

```
*
**
***
****
```

b) Now a centred pyramid of height `n`:

```
   *
  ***
 *****
*******
```

For the pyramid, work out on paper how many spaces and how many stars row i needs *before* you write any code. Two formulas, both in terms of `i` and `n`.

B7 *The largest*

Ask how many numbers there are, then read that many, then print the largest.

You designed this algorithm in English in **week one** — sheet 1, worked example 2. It is the accumulator again, with "bigger than" in place of "plus".

a) What should `largest` start at? Give a reason your answer works even when every number typed is negative.

b) Write it.

c) Now also print *how many* of the numbers were equal to the largest.

If you finish early

Go back to **B7** and try to print the *second* largest number as well.

You will find it is much harder than it looks with one pass and two variables — and almost trivial once you can keep all the numbers somewhere and sort them.

That somewhere is a list, and it is lecture 6.